Computer Lab: SQL Server-6

Data Manipulation(continued)

7.5 Sub queries

We illustrate the SQL statements using the instance of the following tables:

Staff

staffNo	fName	lName	city	position	salary	branchNo
s1	Gad	Ahmed	Tanta	Manager	60,000	B02
s2	Samy	Tamer	Cairo	Assistant	30,000	B02
s3	Samy	Ahmed	Tanta	Supervisor	50,000	B01
s4	Tamer	Samir	Giza	Manager	40,000	B03

Project

projNo	city	staffNo	postcode
PT11	Zakazik	s3	2234
PT21	Demiat	s1	2456
PT32	Tanta	s2	3467

Branch

branchNo	city	telephone
B01	Banha	2234
B02	Cairo	2456
B03	Mansoura	3467

In this section we examine the use of a complete SELECT statement embedded within another SELECT statement.

- The results of this **inner** SELECT statement (or **sub select**) are used in the **outer** statement
- It is used to help determine the contents of the result.
- Sub select can be used in the WHERE and HAVING clauses of an outer SELECT statement
  It is called a **subquery** or **nested query**.
- There are three types of subqueries:
  - A *scalar subquery* returns a single column and a single row; that is, a single value. In principle, a scalar subquery can be used whenever a single value is needed. Example 7.19 uses a scalar subquery.
  - A *row subquery* returns multiple columns, but again only a single row. A row subquery can be used whenever a row value constructor is needed, typically in predicates.
  - A *table subquery* returns one or more columns and multiple rows. A table subquery can be used whenever a table is needed, for example, as an operand for the IN predicate.

Ex 1.1 Using a subquery with equality

List the staff at the branch located in Cairo.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = (SELECT branchNo
                  FROM Branch
                  WHERE city = 'Cairo');
```

- The inner SELECT statement (SELECT branchNo FROM Branch . . .) finds the branch number that corresponds to the branch located in Cairo (there will be only one such branch number, so this is an example of a scalar subquery).
- Having obtained this branch number, the outer SELECT statement then retrieves the details of all staff who work at this branch.
- In other words, the inner SELECT returns a result table containing a single value 'B02', corresponding to the branch at 'Cairo', and the outer SELECT becomes like the coming query.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = 'B02';
```

The result table is shown in Table 1.1.

Table 1.1 Result table for Example 1.1.

staffNo	fName	lName	position
s1	Gad	Ahmed	Manager
s2	Samy	Tamer	Assistant

We can think of the subquery as producing a temporary table with results that can be accessed and used by the outer statement.

- A subquery can be used immediately following a relational operator (=, <, >, <=, > =, < >) in a WHERE clause, or a HAVING clause.
- **The subquery itself is always enclosed in parentheses.**

Ex 1.2 Using a subquery with an Aggregate Function

List all staff whose salary is greater than the average salary and show by how much their salary is greater than the average.

```
SELECT staffNo, fName, lName, position, salary – (SELECT AVG(salary)
FROM Staff) AS salDiff
FROM Staff
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

- First, note that we **cannot write** 'WHERE salary > AVG(salary)' because aggregate functions cannot be used in the WHERE clause.
- Instead, we use a subquery to find the average salary
- And then use the outer SELECT statement to find those staff with a salary greater than this average.
- In other words, the subquery returns the average salary as £45,000.
- Also note the use of the scalar subquery in the SELECT list, to determine the difference from the average salary.

- The outer query is reduced then to:

```
SELECT staffNo, fName, lName, position, salary – 45000 AS salDiff
FROM Staff
WHERE salary > 45000;
```

The result table is shown in Table 1.2.

Table 1.2 Result table for Example 2.2.

staffNo	fName	lName	position	salDiff
s1	Gad	Ahmed	Manager	15,000
s3	Samy	Ahmed	Supervisor	5,000

Example 1.3 Nested subqueries: use of IN

List the projects that are handled by staff who work in the branch located in Cairo.

Below is a selection from the Project table used in the examples:

Project

projNo	city	staffNo	postcode
PT11	Zakazik	s3	2234
PT21	Demiat	s1	2456
PT32	Tanta	s2	3467

```
SELECT projNo, city, postcode
FROM Project
WHERE staffNo IN (SELECT staffNo
                  FROM Staff
                  WHERE branchNo = (SELECT branchNo
                                    FROM Branch
                                    WHERE city = 'Cairo'));
```

Working from the innermost query outwards, the first query selects the number of the branch at Caro city. The second query then selects the staff who work at this branch number. In this case, there may be more than one such row found, and so we cannot use

the equality condition (=) in the outermost query. Instead, we use the IN keyword. The outermost query then retrieves the details of the projects that are managed by each member of staff identified in the middle query. The result table is shown in Table 7.21.

Table 1.3 Result table for Example 1.3.

projNo	city	postcode
PT21	Demiat	2456
PT32	Zakazik	3467

ANY and ALL

The words ANY and ALL may be used with subqueries that produce a single column of numbers.

- If the subquery is preceded by the keyword ALL, the condition will only be true if it is satisfied by all values produced by the subquery.
- If the subquery is preceded by the keyword ANY, the condition will be true if it is satisfied by any (one or more) values produced by the subquery.
- If the subquery is empty, the ALL condition returns true, the ANY condition returns false.
- The ISO standard also allows the qualifier SOME to be used in place of ANY.

Ex 1.4 Use of ANY/SOME

Find all staff whose salary is larger than the salary of at least one member of staff at branch B003.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > SOME (SELECT salary
                     FROM Staff
                     WHERE branchNo = 'B02');
```

While this query can be expressed using a subquery that finds the minimum salary of the staff at branch B02, and then an outer query that finds all staff whose salary is greater than this number (see Example 1.2)

- an alternative approach uses the SOME/ANY keyword.
- The inner query produces the set {60000, 30000}
- and the outer query selects those staff whose salaries are greater than any of the values in this set (that is, greater than the minimum value, 30000).
- This alternative method may seem more natural than finding the minimum salary in a subquery.
- In either case, the result table is shown in Table 1.4 .

Table 1.4 Result table for Example 1.4 .

staffNo	fName	lName	position	salary
s1	Gad	Ahmed	Manager	60,000
s3	Samy	Ahmed	Supervisor	50,000

s4	Tamer	Samir	Manager	40,000

Example 1.5 Use of ALL

Find all staff with salaries larger than the salary of every member of staff whose first name is Samy.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > ALL (SELECT salary
                    FROM Staff
                    WHERE fName = ' Samy');
```

This is very similar to the last example.

- Again, we could use a subquery to find the maximum salary of staff
- and then use an outer query to find all staff whose salary is greater than this number.
- However, in this example we use the ALL keyword.

- The result table is shown in Table 1.5 .

Table 1.5 Result table for Example 1.5 .

staffNo	fName	lName	position	salary
s1	Gad	Ahmed	Manager	60,000

2.1 Multi-Table Queries

All the examples we have considered so far have a major limitation:

- the columns that are to appear in the result table must all come from a single table.
- In many cases, this is not sufficient.
- To combine columns from several tables into a result table we need to use a **join** operation.
  The SQL join operation combines information from two tables by forming pairs of related rows from the two tables.
- The row pairs that make up the joined table are those where the matching columns in each of the two tables have the same value.

If we need to obtain information from more than one table, the choice is between using a subquery and using a join.

- If the result table is to contain columns from different tables, then we must use a join.

- To perform a join, we simply include more than one table name in the FROM clause, using a comma as a separator,
- and typically including a WHERE clause to specify the join column(s).
- It is also possible to use an **alias** for a table named in the FROM clause.
- In this case, the alias is separated from the table name with a space.
    - An alias can be used to qualify a column name whenever there is ambiguity regarding the source of the column name.
    - It can also be used as a shorthand notation for the table name.
    - If an alias is provided it can be used anywhere in place of the table name.

Ex 2.1 Simple join:

Produce a list of all staff and the projects for which they work.

```
SELECT s. staffNo, fName, lName, projNo, p. city
FROM Staff s, Project p
WHERE s. staffNo = p. staffNo;
```

We want to display the details from both the Staff table and the Project table, and so we must use a join.

- Note that it is necessary to qualify the staff number (staffNo) in the SELECT list
- staffNo could come from either table, so we must indicate which one.
- We could equally well have chosen the staffNo column from the Staff table.
- The qualification is achieved by prefixing the column name with the appropriate table name (or its alias).
- In this case, we have used *s* as the alias for the Staff table.

To obtain the required rows, we include those rows from both tables that have identical values in the staffNo columns, using the search condition (c. staffNo = v. staffNo).

We call these two columns the **matching columns** for the two tables.

- The result table is shown in Table 2.1 .

Table 2.1 Result table for Example 2.1 .

staffNo	fName	lName	projNo	city
s1	Gad	Ahmed	PT21	Demiat
s2	Samy	Tamer	PT32	Tanta
s3	Samy	Ahmed	PT11	Zakazik

In Example 2.1 , we compared the primary key in the Staff table, s. staffNo, with the foreign key in the Project table, p. staffNo.

Example 7.2 Sorting a join

Produce a list of all staff and the projects for which they work, *arranged in order of project number and the staff number.*

```
SELECT projNo, s. staffNo, fName, lName, p. city
FROM Staff s, Project p
WHERE s. staffNo = p. staffNo;
ORDER BY projNo, s.staffNo;
```

To make the results more readable, we have ordered the output using the project number as the major sort key and the staff number as the minor key. The result table is shown in Table 7.2 .

Table 7.2 Result table for Example 7.2 .

projNo	staffNo	fName	lName	city
PT11	s3	Samy	Ahmed	Zakazik
PT21	s1	Gad	Ahmed	Demiat
PT32	s2	Samy	Tamer	Tanta

Example 7.3 Three-table join

For each branch, list the numbers and names of staff who work in projects, including the city in which the branch is located and the project number that the staff manages.

```
SELECT b.branchNo, b.city, s.staffNo, fName, lName, projectNo
FROM Branch b, Staff s, Project p
WHERE b.branchNo = s.branchNo AND s.staffNo = p.staffNo
ORDER BY b.branchNo, s.staffNo, projectNo;
```

The result table requires columns from three tables: Branch, Staff, and Project, so a join must be used. The Branch and Staff details are joined using the condition (b.branchNo = s.branchNo), to link each branch to the staff who work there. The Staff and Project details are joined using the condition (s.staffNo = p.staffNo), to link staff to the projects they work in. The result table is shown in Table 7.3.

Table 7.3 Result table for Example 7.3.

branchNo	city	telephone	staffNo	fName	lName	projNo
B01	Banha	2234	s3	Samy	Ahmed	PT11
B02	Cairo	2456	s1	Gad	Ahmed	PT21
B02	Cairo	2456	s2	Samy	Tamer	PT32